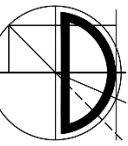


Grundlagen der Programmierung

Kapitel 8: Rekursion

Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich

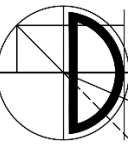


- Abgrenzung
- Definition und Beispiele
- Datenstruktur Stack
- Aufrufbaum
- Rekursion als Problemlösungsstrategie
- Modulare Programmierung



“To iterate is human, to recurse, divine.”

-- anonymous

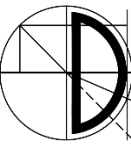


■ Iteration

- Verfahren, das ein Problem schrittweise **durch wiederholte Anwendung desselben Verfahrens-Schritts** löst. Jeder dieser Iterationsschritte liefert Ergebnisse, die im nächsten Iterationsschritt weiter verwendet werden.
- Abbruch der Iteration, wenn die Ergebnisse eines Iterationsschrittes eine Bedingung erfüllen.
- Im Programm: **Schleife**

■ Rekursion

- Verfahren, das ein Problem **durch Zurückführen auf ein einfacheres Problem** löst. Für die Lösung des einfacheren Problems greift das Verfahren **auf sich selbst** zurück.
- Abbruch der Rekursion, wenn das einfachere Problem gelöst werden kann, ohne dass erneut auf das Verfahren selbst zurückgegriffen werden muss.
- Im Programm: **Rekursive Methode**



Iterativ

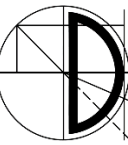
$$n! = \begin{cases} 1 & \text{für } n = 0 \\ 1 \cdot 2 \cdot \dots \cdot n & \text{für } n > 0 \end{cases}$$

```
static long fakul(long n) {  
    long f = 1;  
    for (long i=1; i<=n; i++)  
        f = f * i;  
    return f;  
}
```

Rekursiv

$$n! = \begin{cases} 1 & \text{für } n = 0 \\ n \cdot (n-1)! & \text{für } n > 0 \end{cases}$$

```
static long fakul(long n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * fakul(n-1);  
}
```

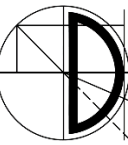


- **Iterativer Algorithmus**

- Bei der Ausführung werden bestimmte Teile des Algorithmus mehrfach ausgeführt

- **Rekursiver Algorithmus**

- Bei der Ausführung wird in bestimmten Teilen des Algorithmus der Algorithmus selbst direkt oder indirekt aufgerufen
- **Vorteil** von rekursiven Algorithmen: Elegant und kurz für viele Problemstellungen
- **Nachteil** von rekursiven Algorithmen: Manchmal ineffizient
- *Die rekursive Formulierung kann immer in eine äquivalente iterative Formulierung umgewandelt werden*
 - Explizite Auswertung und Verwaltung des *Aufruf-Stacks* (siehe Folgefolien)



- Bei der Ausführung der Methode wird die Methode selbst direkt oder indirekt wieder aufgerufen.

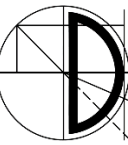
direkt: Im Rumpf der Methode **m** steht ein Aufruf der Methode **m**

Indirekt: Im Rumpf der Methode **m** steht ein Aufruf der Methode **a**
Im Rumpf der Methode **a** steht der Aufruf der Methode **b**
Im Rumpf der Methode **b** steht der Aufruf der Methode ...
...
Im Rumpf der Methode .. steht der Aufruf der Methode **m**

- Weiteres Beispiel: Einfache Potenzfunktion (vgl. **Math.pow**)

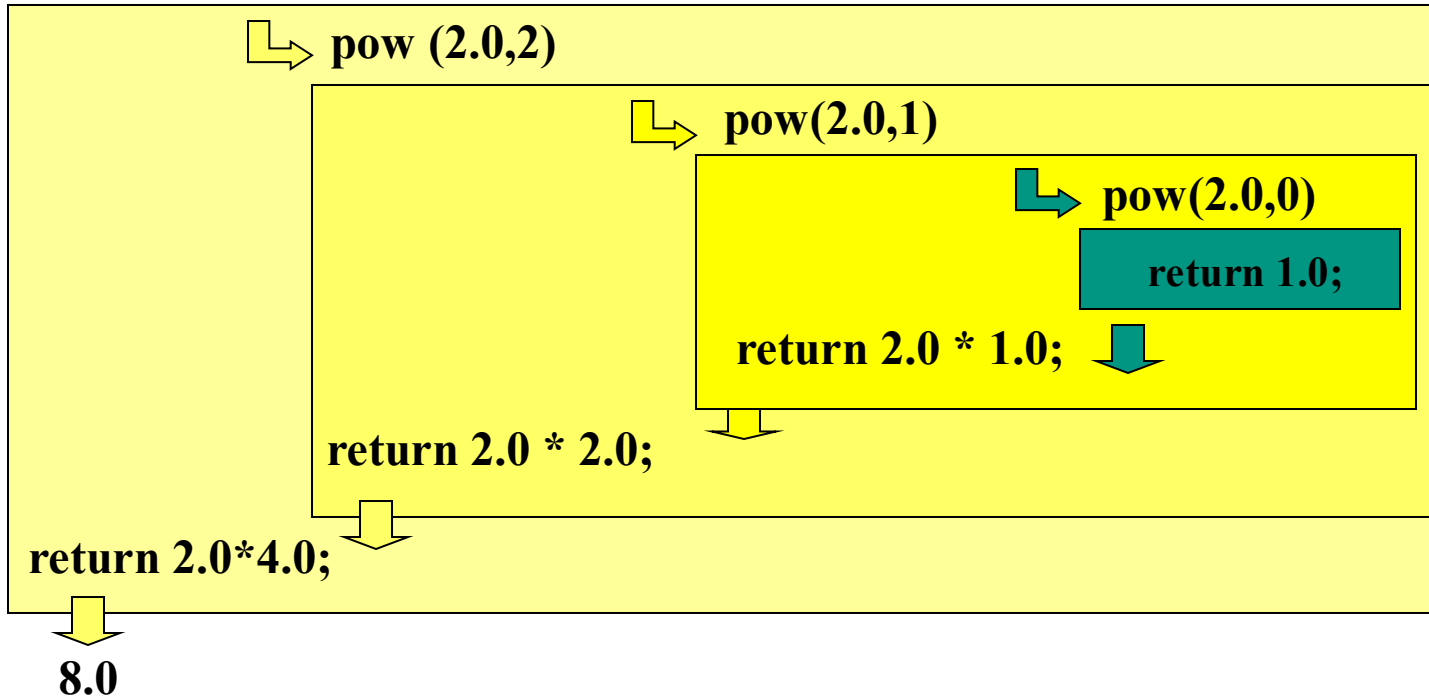
$$x^k = \begin{cases} 1 & \text{für } k = 0 \\ x \cdot x^{k-1} & \text{für } k > 0 \\ 1 / x^{-k} & \text{für } k < 0 \end{cases}$$

```
static double pow(double x, int k) {  
    if (k == 0)  
        return 1;  
    else if (k > 0)  
        return x * pow(x, k-1);  
    else  
        return 1 / pow(x, -k);  
}
```

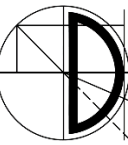


Ablauf des Methodenaufrufs:

`pow(2.0,3)`



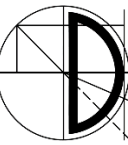
```
static double pow(double x, int k) {  
    if (k == 0)  
        return 1;  
    else if (k > 0)  
        return x * pow(x, k-1);  
    else  
        return 1 / pow(x, -k);  
}
```

```
public static int summeVon1bis(int n) {  
    return n + summeVon1bis(n-1);  
}
```

Laufzeitfehler: Endlos-Rekursion!

- Wichtig!
 - Rekursionen müssen abbrechen
 - Ein oder mehrere einfache Fälle müssen gesondert behandelt werden
- Grundregel bei der Implementierung
 - Erst die nichtrekursiven einfachen Fälle implementieren
 - Danach erst den Rekursionsschritt programmieren



- Fibonacci-Zahlen und Kaninchenvermehrung
- Leonardo Pisano **Fibonacci** formulierte 1190:

Wenn ein neugeborenes Kaninchenpaar nach 2 Monaten ein neues Kaninchenpaar wirft und dann monatlich jeweils ein weiteres Paar und außerdem jedes neugeborene Paar sich auf die gleiche Art vermehrt, wie viele Kaninchenpaare gibt es dann nach n Monaten, wenn keines der Kaninchen vorher stirbt?

- Anzahl Kaninchenpaare nach n Monaten: **fib**(n)

$$\text{fib}(0) = 1$$

$$\text{fib}(1) = 1$$

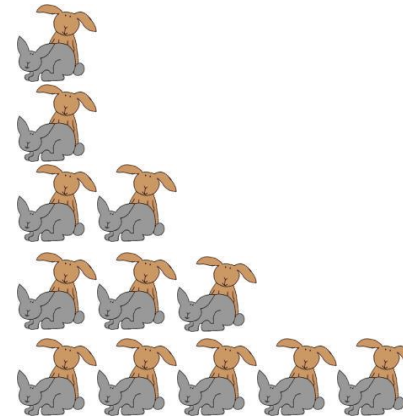
$$\text{fib}(2) = 1 + 1 = 2$$

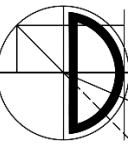
$$\text{fib}(3) = 2 + 1 = 3$$

$$\text{fib}(4) = 3 + 2 = 5$$

...

$$\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$$





- Rekursive Fibonacci-Methode (sehr ineffizient!)

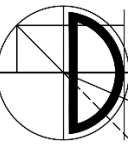
```
static long fib(long n) {  
    if(n == 0)  
        return 1;  
    else if(n == 1)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

n = 40
rekursiv fib(40) = 165580141
Laufzeit: 3157 ms

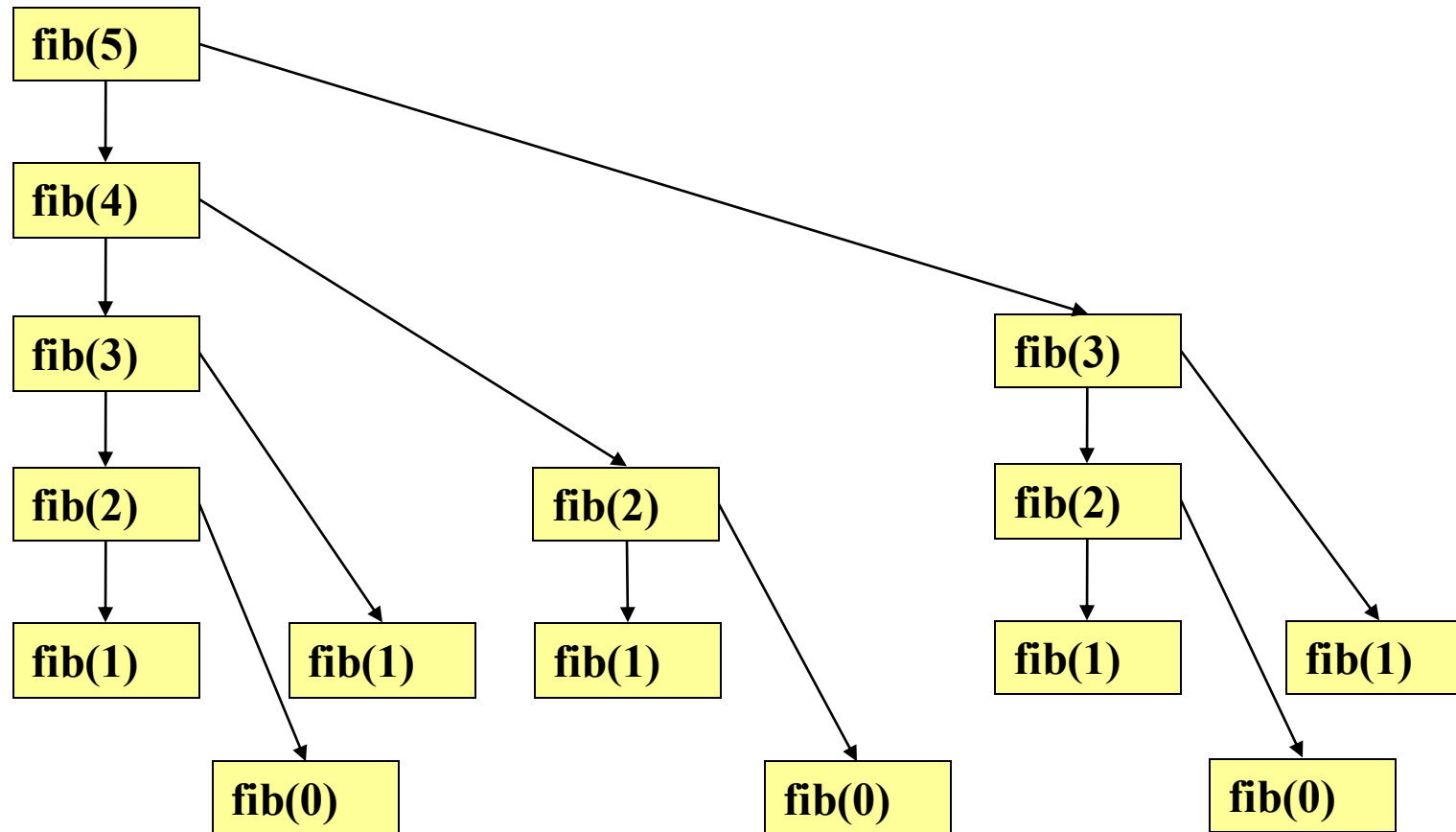
- Iterative Fibonacci-Methode

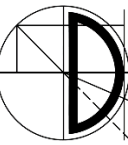
```
static long fib(long n) {  
    long fN = 1, fNminus1 = 0, fNminus2;  
    for(long i=0; i<n; i++) {  
        fNminus2 = fNminus1;  
        fNminus1 = fN;  
        fN = fNminus1 + fNminus2;  
    }  
    return fN;  
}
```

n = 40
iterativ fib(40) = 165580141
Laufzeit: <1 ms



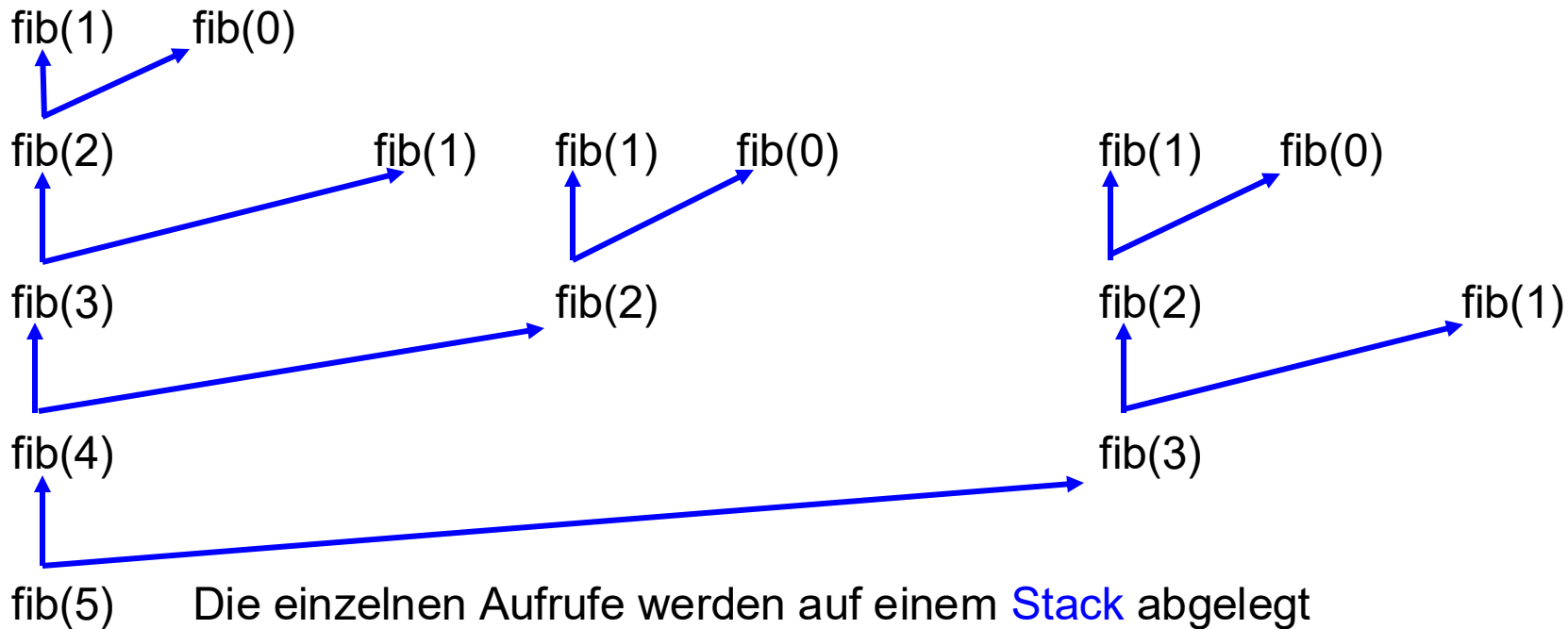
Ablauf des rekursiven Methodenaufrufs am Beispiel `fib(5)`

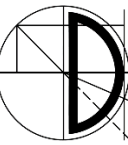




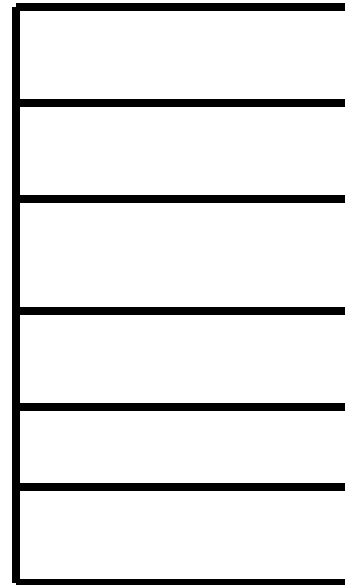
```
static long fib(long i) {  
    if (i == 0) return 1;  
    else if (i == 1) return 1;  
    else return fib(i-1) + fib(i-2);  
}
```

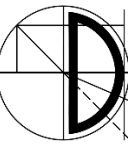
Der **Aufrufbaum** der Methode für den Fall Fib(5) (gedreht analog zum Stack):





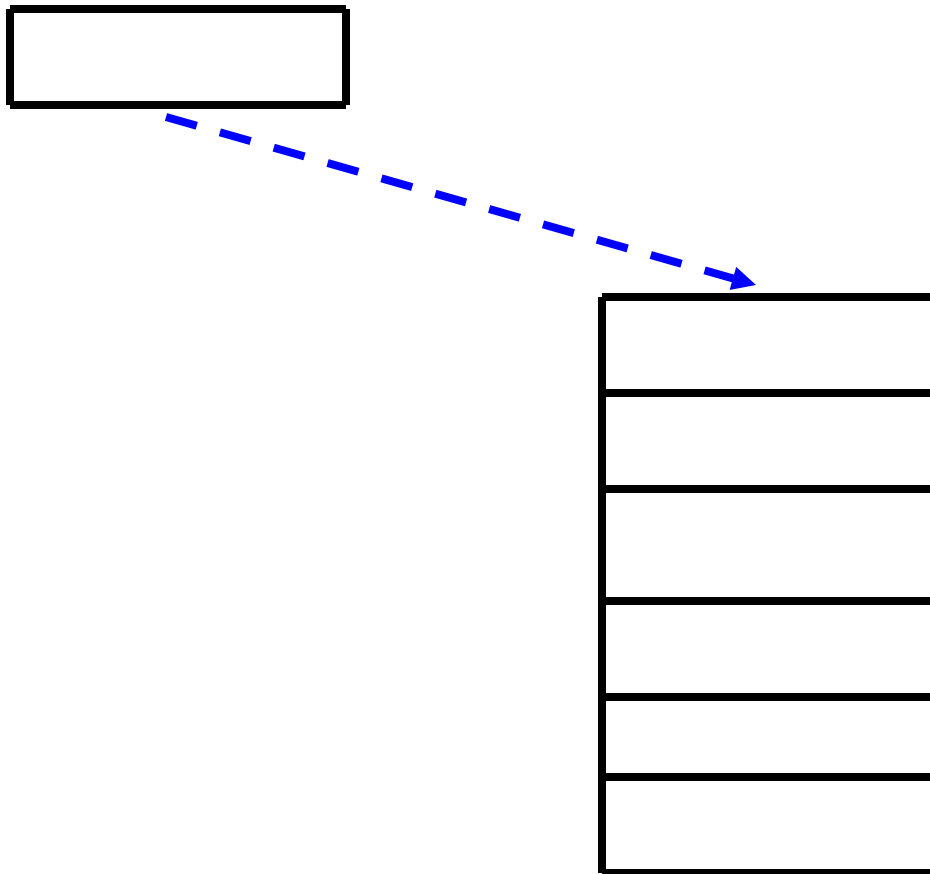
Datenstruktur: **Stack** (Stapel)

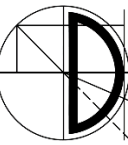




Datenstruktur: **Stack**

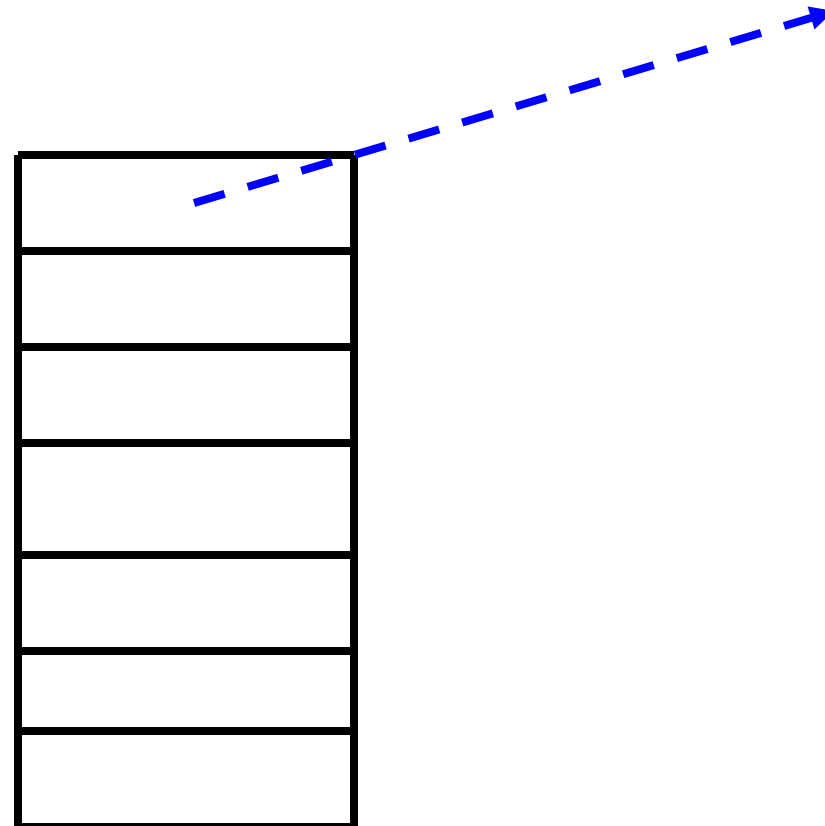
push - Ablegen eines neuen Elements auf dem Stack

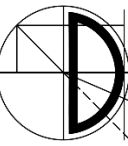




Datenstruktur: **Stack**

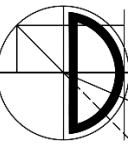
pop - Entfernen des obersten Elements vom Stack





- Weitere mögliche Operationen:
 - Erzeugen eines neuen Stacks (**new**), Lesen des Inhalts des obersten Elements (**top**), Prüfen, ob der Stack leer ist (**empty**)





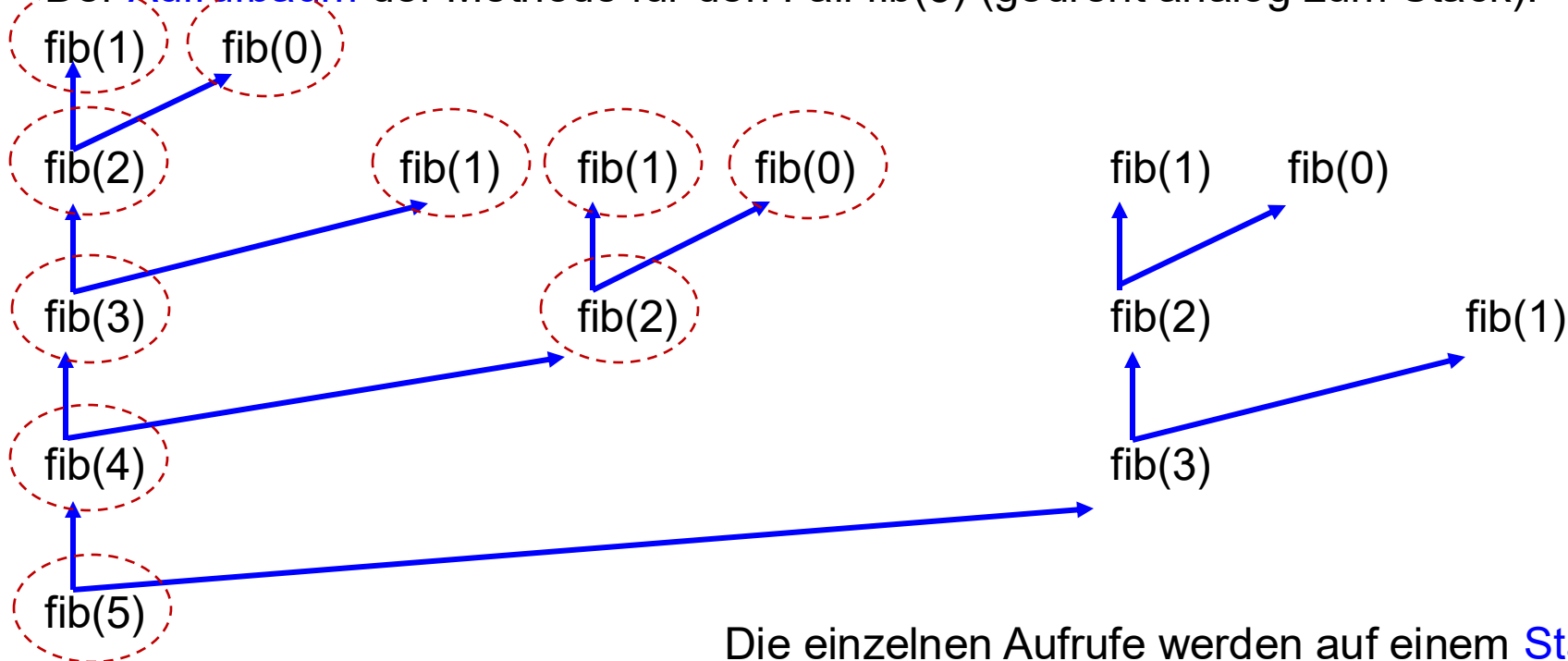
- Datenstruktur: **Stack**
- Ein Stack realisiert das **LIFO**-Prinzip: **L**ast **I**n **F**irst **O**ut
- Praktische Realisierungsmöglichkeiten:
 - als Array (bei bekannter oberer Schranke für die Anzahl der Elemente)
 - als linear verkettete Liste (siehe später dynamische Datenstrukturen, Listen, binäre Bäume)

```
static long fib(long i) {
    if (i == 0) return 1;
    else if (i == 1) return 1;
    else return fib(i-1) + fib(i-2);
}
```

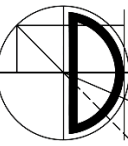
Stack

0
0
2
4
5

Der **Aufrufbaum** der Methode für den Fall fib(5) (gedreht analog zum Stack):

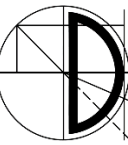


Die einzelnen Aufrufe werden auf einem **Stack** abgelegt

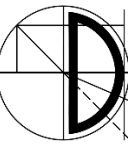


- Die rekursive Formulierung eines Problems kann immer in eine äquivalente iterative Formulierung umgewandelt werden
 - explizite Auswertung und Verwaltung des **Prozedurstacks (Aufruf-Stacks)**
- **Rekursion** liefert eine wichtige **allgemeine Methode** zur Lösung algorithmischer Probleme:

```
loese rekursiv ( Problem ) {  
    if (Lösung des Problems trivial) { return Lösung }  
    else {  
        zerlege Problem in Teilprobleme;  
        löse rekursiv ( Teilprobleme );  
        setze die Lösungen der Teilprobleme  
        zu einer Lösung des Gesamtproblems zusammen;  
    }  
}
```



- Durch Methoden wird ausführbarer Code unter einem Namen zusammengefasst
- Dieser Code kann unter Verwendung von Parametern formuliert sein, denen später beim Aufruf der Methode Werte übergeben werden
- Die Übergabe der Parameter ist **Pass-by-Value / Call-by-Value**
- Methoden erlauben die Modularisierung des Programms durch die Abtrennung von Teilaufgaben, die in eigenständigen Einheiten zusammengefasst werden
- Um die Wiederverwendbarkeit (**reusability**) zu unterstützen sollte jede Methode möglichst nur eine einzelne, wohldefinierte Aufgabe haben (Vermeidung von Nebeneffekten) und einen Namen tragen, der die Aufgabe zum Ausdruck bringt
- Wenn dies nicht geht, so sollte man überlegen, ob man die Methode nicht in kleinere, überschaubarere zerlegt



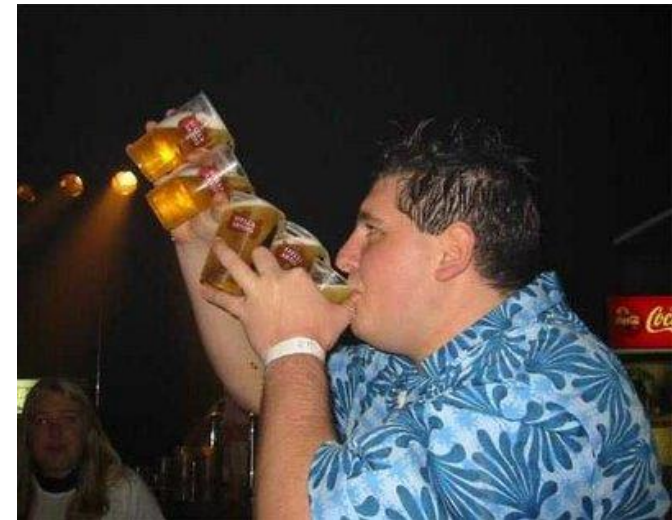
- Man unterscheidet Prozeduren und Funktionen:
 - Prozeduren liefern **keinen** Rückgabewert
 - Funktionen liefern einen Rückgabewert

An jeder Stelle, an der ein bestimmter Wert verwendet werden kann, kann auch ein Methodenaufruf verwendet werden, der diesen Wert zurückliefert!

Regel 8 (der Goldenen Regeln der Code-Formatierung): Eine Methode mit einem Rückgabewert heißt Funktion. Eine Funktion besitzt stets nur **eine einzige return-Anweisung**. Diese steht in der letzten Zeile des Methodenblocks und lautet **return result;**

Schon gewusst, was stackoverflow.com und Rekursion verbindet?

Die populäre Webseite Stack Overflow hat ihren Namen von dem Fehler, den man unter anderem bekommt, wenn man eine unendliche/sehr tiefe Rekursion ausführt. Für jeden Methodenaufruf wird Arbeitsspeicher (z.B. für die Methodenparameter) verbraucht. Wenn der begrenzte Speicher des Stacks „überläuft“, tritt der Fehler auf und führt zum Absturz des Programms. Stack Overflows können z.B. auch durch kurz sehr große Variablen entstehen.



The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a large, bold, white sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit